New lower bounds for trifferent codes of lengths 10, 11, 12, 16 and 17

Tyler Duic
Independent researcher, New York, NY
https://github.com/tduic

October 2026

Abstract

A code $C \subseteq \{0,1,2\}^n$ is trifferent if any three distinct codewords have a coordinate in which they are pairwise distinct; equivalently, C is a perfect hash family $\mathrm{PHF}(n; |C|, 3, 3)$. Writing $T(n)$ for the largest size of a trifferent code of length n, we give explicit codes showing $T(10) \geq 30$, $T(11) \geq 36$, $T(12) \geq 48$, $T(16) \geq 108$ and $T(17) \geq 109$. Each improves on every bound we found in the literature, including the closure of the known values under the elementary monotonicity and shortening rules. The codes have short algebraic descriptions: unions of orbits of a translation and a cyclic shift for $n \leq 12$, and four cosets of a linear $[16, 3]_3$ code for $n = 16$. Each bound is verified by the Lean 4 proof assistant against a direct definition of trifference.

1 Introduction

A code $C \subseteq \{0,1,2\}^n$ is *trifferent* if for any three distinct $x, y, z \in C$ there is a coordinate i with $\{x_i, y_i, z_i\} = \{0,1,2\}$. Let $T(n)$ denote the maximum size of a trifferent code of length n. Determining the growth rate of $T(n)$ is a long-standing problem in information theory and combinatorics; the classical bounds are due to Fredman and Komlós [1] and Körner and Marton [2]. Equivalently, a trifferent code of length n and size k is a perfect hash family $\mathrm{PHF}(n; k, 3, 3)$: the n coordinates are functions from the k codewords to a 3-set, and every 3-subset of codewords is separated by one of them [3].

Exact values are known for small lengths: $T(5) = 10$ and $T(6) = 13$ [4], and $T(7) = 16$, $T(8) = 20$, $T(9) = 27$ [5]. For larger n, lower bounds come from tables of perfect hash families [3] and from linear codes: a linear code over $\mathbb{F}_3$ is trifferent if and only if it is minimal, and minimal $[14, 4]_3$ and $[19, 5]_3$ codes give $T(14) \geq 81$ and $T(19) \geq 243$ [5, 6].

Our contribution is five improved lower bounds for small lengths, each witnessed by an explicit code and checked by a proof assistant.

Theorem 1. *$T(10) \geq 30$, $T(11) \geq 36$, $T(12) \geq 48$, $T(16) \geq 108$ and $T(17) \geq 109$.*

2 The baseline

Two elementary rules propagate bounds between lengths. First, $T(n) \geq T(n-1) + 1$ [5, Lemma 1 and Corollary 2]: given a trifferent code C of length $n-1$ and $c \in C$, the code $\{(c,0), (c,1)\} \cup \{(c', 2) : c' \in C \setminus \{c\}\}$ is trifferent of length n. Second, *shortening*: $T(n-1) \geq \lceil 2T(n)/3 \rceil$, equivalently

$T(n) \le \lfloor 3T(n-1)/2 \rfloor$ (cf. [5]). Indeed, if C has length n, some symbol a occurs in a given coordinate in at most $|C|/3$ codewords; deleting those codewords and then that coordinate leaves a trifferent code of length $n-1$ with at least $\lceil 2|C|/3 \rceil$ words. Three remaining words take at most two values in the deleted coordinate, so they are separated by another coordinate; for the same reason (when at least three words remain) no two remaining words coincide after the deletion.

As a baseline we take the best published values—the exact values above, the linear-code bounds $T(14) \ge 81$ and $T(19) \ge 243$, and the best values in Table 6 of Walker and Colbourn [3], which are $29, 30, 36, 37, 48, 57, 81, 82, 83$ for $n = 10, \dots, 18$—and close them under both rules. Table 1 compares the result with Theorem 1.

n	baseline	source of baseline	new bound	gain
10	29	Walker–Colbourn [3]	30	+1
11	30	Walker–Colbourn [3]	36	+6
12	36	Walker–Colbourn [3]	48	+12
16	83	$T(14) \ge 81$ and two steps of $T(n) \ge T(n-1)+1$	108	+25
17	108	shortening $T(18) \ge 162$, itself shortened from $T(19) \ge 243$	109	+1

Table 1: Best bounds we found before this work (the closure described in the text) and the new bounds.

Remark 2. We also found codes of size 54 at $n = 13$ and 162 at $n = 18$, but these sizes equal the shortenings of the minimal $[14, 4]_3$ and $[19, 5]_3$ codes, so they are not new bounds.

3 The codes

Throughout, codewords are written as strings over $\{0, 1, 2\}$, and for $s \mid n$ we let G_s be the group of maps on $\{0, 1, 2\}^n$ generated by $x \mapsto x + (1, \dots, 1) \pmod 3$ and the cyclic shift of coordinates by s positions.

Length 10. The code is the union of the G_2-orbits of `0001101220` and `0012020120`, each of size 15, giving 30 words.

Length 11. The code is the union of the G_1-orbits of `00000000000` (size 3) and `00011101201` (size 33), giving 36 words.

Length 12. The code is the union of the G_2-orbits of `000111222111`, `000121012221` (each of size 18), `000122201112` and `010201020102` (each of size 6), giving 48 words.

Length 16. Let $D \subseteq \mathbb{F}_3^{16}$ be the linear code spanned by

`0001221202201001`, `0110120102221111`, `1010100010111101`.

It has dimension 3 and nonzero weights 9 and 12; being trifferent and linear, it is a minimal code [6]. The code is the union of the four cosets $r + D$ with

$r \in \{$`0000000000000000`, `0010011222101021`, `0010122101011012`, `0020221012102011`$\}$,

giving 108 words.

Length 17. Apply the construction of [5, Lemma 1] to the length-16 code with $c = 0$: the code is $\{(0,0),(0,1)\} \cup \{(c',2) : c' \in C_{16} \setminus \{0\}\}$, where C_{16} is the length-16 code, with 109 words.

The full codes are listed in machine-readable form in the accompanying repository, together with a script that rebuilds them from the descriptions above.

4 Verification

Checking trifference directly is a finite computation over $\binom{|C|}{3}$ triples—about 2×10^5 for the length-16 code—but we wanted a check that does not rely on trusting a particular program. We therefore verified each bound in the Lean 4 proof assistant [7] using Mathlib [8].

- **Definition.** Trifference is defined directly on finite sets of functions $\{1,\dots,n\} \to \{0,1,2\}$: for all pairwise distinct $x, y, z \in C$ there is i with x_i, y_i, z_i pairwise distinct.
- **Statement.** Each bound in Theorem 1 is proved in the form $\exists\, C \subseteq \{0,1,2\}^n,\ C$ trifferent $\wedge\ k \le |C|$.
- **Method.** A bitmask checker on lists of codewords is proved sound once, in general; it is then evaluated on each explicit code by the Lean kernel (using `decide` with kernel reduction, in chunks to bound memory). No compiled or native evaluation is trusted, and the only axioms used are the standard ones of Lean's logic (`propext`, `Classical.choice`, `Quot.sound`).
- **Independent statement check.** A separately written definition of trifference is proved equivalent to the one used, each bound is restated with it, and a three-word code that is not trifferent is shown to be rejected, so the definition is not vacuous.

Independently of Lean, every code was re-checked triple by triple with separate code, and the Lean certificates were decoded and compared word for word with the published codes.

5 Remarks

The gains at $n = 10$ and $n = 17$ are small; the latter is a direct consequence of the length-16 code. The largest gains are at $n = 12$ and $n = 16$, where structured codes—orbit unions and cosets of a minimal linear code—exceed both the values in the perfect hash family tables and the linear-code bounds propagated by the elementary rules. We compared against the trifferent-code literature cited above and the perfect hash family tables of [3]. We were unable to consult later online tables of perfect hash families and would be grateful to learn of any entry there that matches or exceeds these values.

Data and code availability. The codes, the Lean 4 proofs and the checking scripts are at `https://github.com/tduic/research-results/tree/main/trifferent-codes`.

Acknowledgments. The search, the Lean formalization and the independent checks were carried out with substantial assistance from AI models (Anthropic's Claude), working under the author's direction; the author checked every claim and reference in this note and takes full responsibility for its content.

References

[1] Michael L. Fredman and János Komlós. On the size of separating systems and families of perfect hash functions. *SIAM Journal on Algebraic and Discrete Methods*, 5(1):61–68, 1984. doi: 10.1137/0605009.

[2] János Körner and Katalin Marton. New bounds for perfect hashing via information theory. *European Journal of Combinatorics*, 9(6):523–530, 1988. doi: 10.1016/S0195-6698(88)80048-9.

[3] Robert A. Walker, II and Charles J. Colbourn. Perfect hash families: Constructions and existence. *Journal of Mathematical Cryptology*, 1(2):125–150, 2007. doi: 10.1515/JMC.2007.008.

[4] Stefano Della Fiore, Alessandro Gnutti, and Sven C. Polak. The maximum cardinality of trifferent codes with lengths 5 and 6. *Examples and Counterexamples*, 2:100051, 2022. doi: 10.1016/j.exco.2022.100051.

[5] Sascha Kurz. Trifferent codes with small lengths. *Examples and Counterexamples*, 5:100139, 2024. doi: 10.1016/j.exco.2024.100139. arXiv:2310.13563.

[6] Anurag Bishnoi, Jozefien D'haeseleer, Dion Gijswijt, and Aditya Potukuchi. Blocking sets, minimal codes and trifferent codes. *Journal of the London Mathematical Society*, 109(6):e12938, 2024. doi: 10.1112/jlms.12938. arXiv:2301.09457.

[7] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[8] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.